

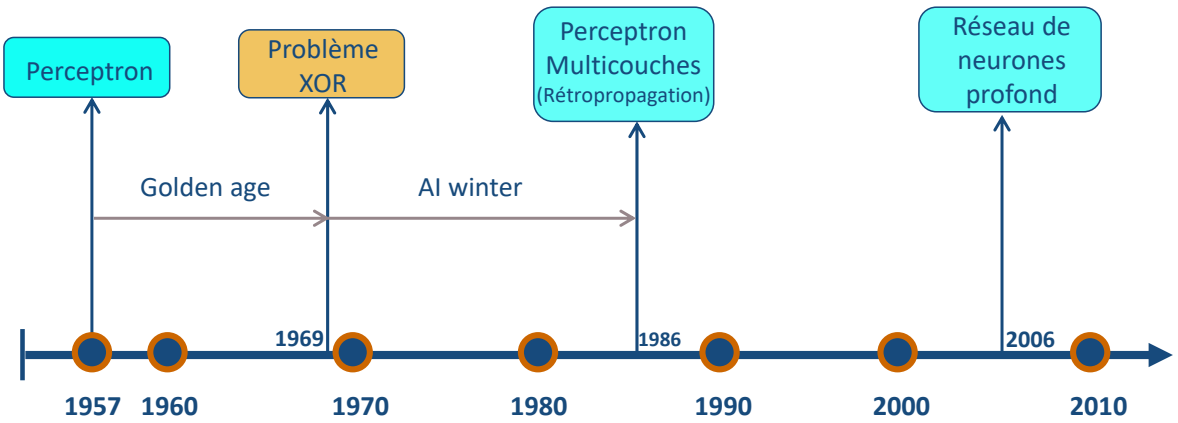
CHAPITRE 1

APPRENTISSAGE DES RÉSEAUX DE NEURONES PROFONDS

T. Ben Said & T. Ben Abdallah

IIT-Sfax

Historique du développement des réseaux de neurones

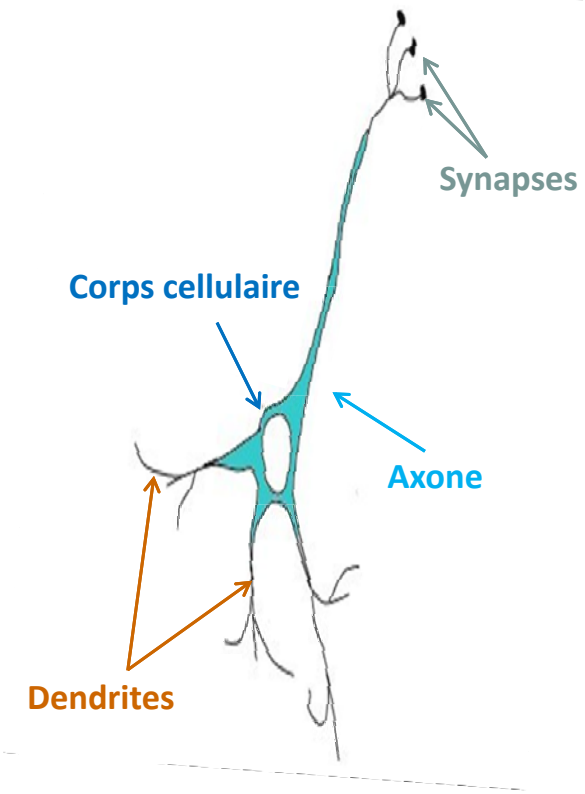


PERCEPTRON SIMPLE

T. Ben Said & T. Ben Abdallah

IIT-Sfax

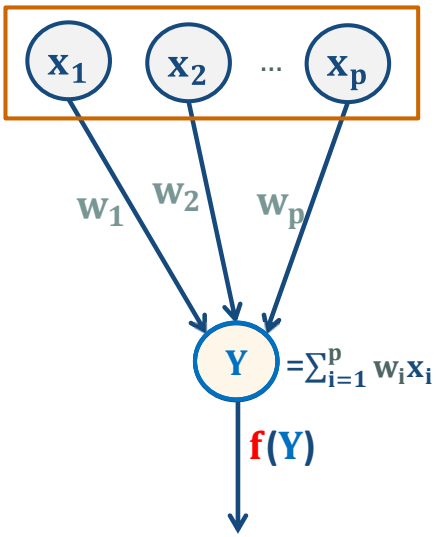
Neurone biologique



- Transitent l'information venue de l'extérieur vers le corps cellulaire
- **centre de contrôle** Fait la somme des informations qui lui parviennent
- Traite l'information et renvoie le résultat sous forme de signaux électriques, du corps cellulaire à l'entrée des autres neurones
- Points de contrôle entre deux neurones: ils permettent aux neurones de communiquer entre eux

$\simeq 10^{11}$ neurones dans le cerveau humain
$\simeq 10^{4}$ connexions (synapses + axones) / neurone

Neurone Artificiel



x_1	x_2	...	x_p	Classe_d

Dendrites : Série d'entrées $x_1 \ldots x_p$

Synapses : coefficients synaptiques $w_1 \ldots w_p$

=> Chaque entrée est associée à un poids

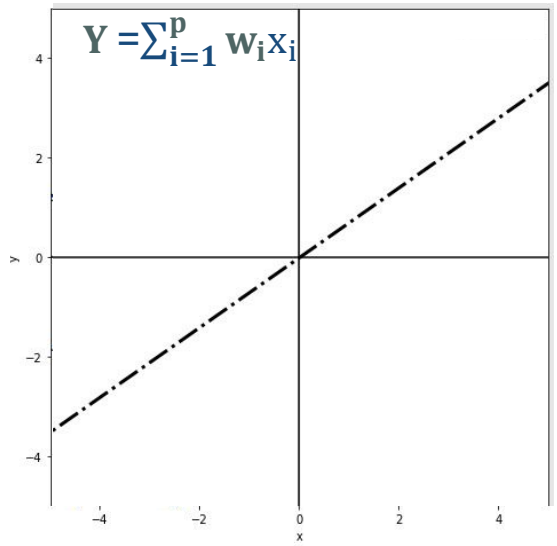
Corps cellulaire : Traitement (calcul de Y)

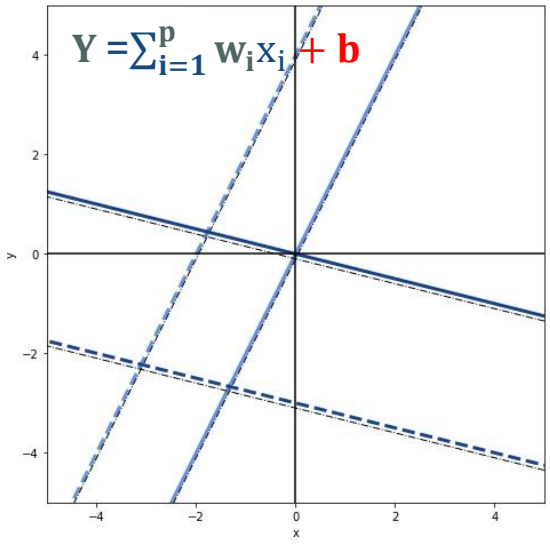
Axone : Fonction d'activation **f**

=> Résultat de neurones = sortie

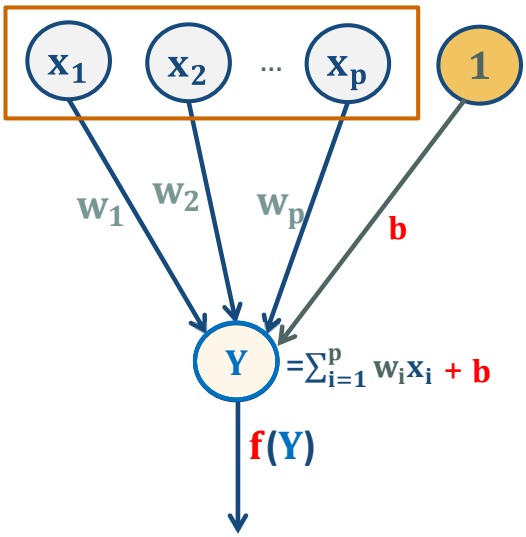
Neurone Artificiel

Biais b





Neurone Artificiel



x_1	x_2	...	x_p	Classe_d

Dendrites : Série d'entrées $x_1 \ldots x_p$

Synapses : coefficients synaptiques $w_1 \ldots w_p$

=> Chaque entrée est associée à un poids

Corps cellulaire : Traitement (calcul de Y)

Axone : Fonction d'activation **f**

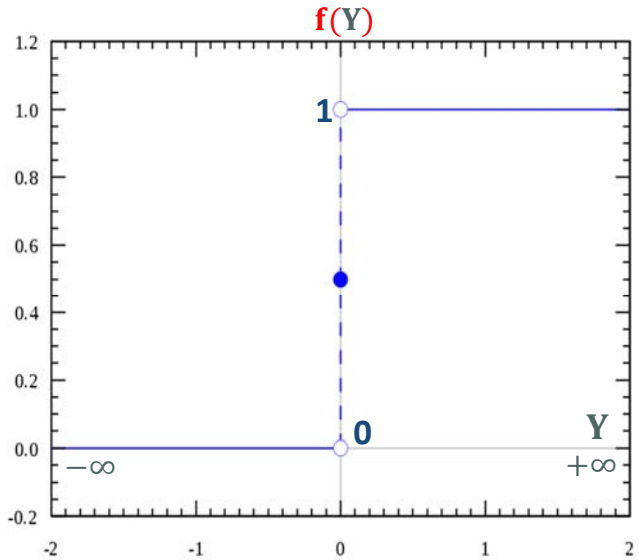
=> Résultat de neurones = sortie

Biais b =Régulateur

=> Aide à s'adapter au mieux aux données

Neurone Artificiel

Fonction d'activation = Heaviside (Marche)



$Y = \sum_{i=1}^{p} w_i x_i + b$

$f(Y)=1$ si $Y \geq 0$

$f(Y)=0$ sinon

Deux valeurs : **0** ou **1**

☑ Classification **binaire**

☑ Problème linéairement séparable!

Neurone Artificiel

Apprentissage = Ajustement des poids

Loi de Widrow-Hoff (Delta rule)

Erreur (loss)

$$w_i = w_i + \eta(d - \text{sortie})x_i$$

- η : Pas d'apprentissage = **Learning rate** $\epsilon\ ]0..1]$
  Contrôle la rapidité avec laquelle le modèle est adapté au problème
- d : Classe désirée
- sortie : Classe prédite = $f(Y)$

Si $\text{sortie}=d$, w_i sont inchangeables

Si $\text{sortie}<d$, w_i va augmenter

Si $\text{sortie}>d$, w_i va diminuer

Exemple introductif (1/4)

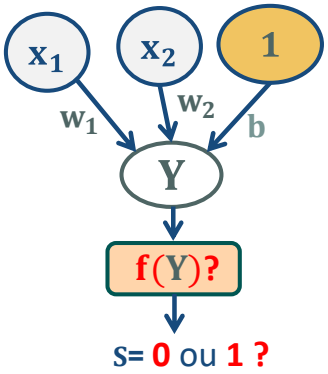
Étant donné le jeu de donnée d'apprentissage `or_logique`

Variable à prédire (cible) : **d**→ classification binaire (0/ 1)

Variables explicatives : x_1, et x_2

	x1	x2	d
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	1

* Représenter **l'architecture du perceptron** associé à `or_logique`?



Exemple introductif (2/4)

Objectif : ajuster les poids w_1, w_2, et b !

Etapes à suivre !

1- Initialiser aléatoirement les poids et Bias w_1, w_2, et b

$w_1 = 1, w_2 = -1, b = 0$

	w_1	w_2	b	x_1	x_2	d	Y	f(Y)	w_1	w_2	b
0	-	-	-	-	-	-	-	-	1	-1	0

2- Faire passer la première observation ($x_1 = 0; x_2 = 0$; d=0) sur le réseau et mettre à jour w_1, w_2, et b sachant que η=1

$Y = \sum_{i=1}^{2} w_i x_i + b = w_1 x_1 + w_2 x_2 + b = (1\times0)+(-1\times0)+0=0$

$f(Y)$=1 puisque $Y \geq 0$

$w_1 = w_1 + \eta(d - sortie)x_1 = w_1 + \eta(d - f(Y))x_1 = 1+1(0-1)0=1$

	w_1	w_2	b	x_1	x_2	d	Y	f(Y)	w_1	w_2	b
1	1	-1	0	0	0	0	0	1	1	-1	-1

Exemple introductif (3/4)

3- Faire passer les trois observations restantes une par une sur le réseau et mettre à jour w_1, w_2, et b sachant que η=1

	w_1	w_2	b	x_1	x_2	d	Y	f(Y)	w_1	w_2	b
2	1	-1	-1	0	1	1	-2	0	1	0	0
3	1	0	0	1	0	1	1	1	1	0	0
4	1	0	0	1	1	1	1	1	1	0	0

4-

Répéter

> Repasser les observations une par une sur le réseau et mettre à jour w_1, w_2, et b sachant que η=1

Jusqu'à aucune correction n'est effectuée en passant toutes les observations (convergence)

Exemple introductif (4/4)

	w_1	w_2	b	x_1	x_2	d	Y	f(Y)	w_1	w_2	b
0	-	-	-	-	-	-	-	-	1	-1	0
1	1	-1	0	0	0	0	0	1	1	-1	-1
2	1	-1	-1	0	1	1	-2	0	1	0	0
3	1	0	0	1	0	1	1	1	1	0	0
4	1	0	0	1	1	1	1	1	1	0	0
5	1	0	0	0	0	0	0	1	1	0	-1
6	1	0	-1	0	1	1	-1	0	1	1	0
7	1	1	0	1	0	1	1	1	1	1	0
8	1	1	0	1	1	1	2	1	1	1	0
9	1	1	0	0	0	0	0	0	1	1	-1
10	1	1	-1	0	1	1	0	1	1	1	-1
11	1	1	-1	1	0	1	0	1	1	1	-1
12	1	1	-1	1	1	1	1	1	1	1	-1
13	1	1	-1	0	0	0	-1	0	1	1	-1

ÉPOQUE (EPOCH)

Conditions d'arrêt

- Les poids sont stables→ aucune mise à jour n'est effectuée en passant une époque
- L'erreur globale ne diminue plus significativement
- Le taux d'erreur atteint un minimum fixé (seuil)
- Le nombre d'itérations atteint un maximum fixé
- Le nombre d'époques est atteint

Activité

Étant donné un jeu de données d'apprentissage de **3** descripteurs numériques $\mathbf{n_1}$, $\mathbf{n_2}$,et $\mathbf{n_3}$ et une classe **d** (**oui** et **non**)

Le passage par **deux époques** (en respectant l'ordre des observations) sur un réseau de neurones de deux couches (entrée et sortie) engendre dans les **10** dernières itérations les valeurs de poids suivants : $\mathbf{b}$= -0.3,$\mathbf{w_1}$=0.2, $\mathbf{w_2}$= 2 et $\mathbf{w_3}$= -0.1

1- Calculer la sortie de la 21ème itération

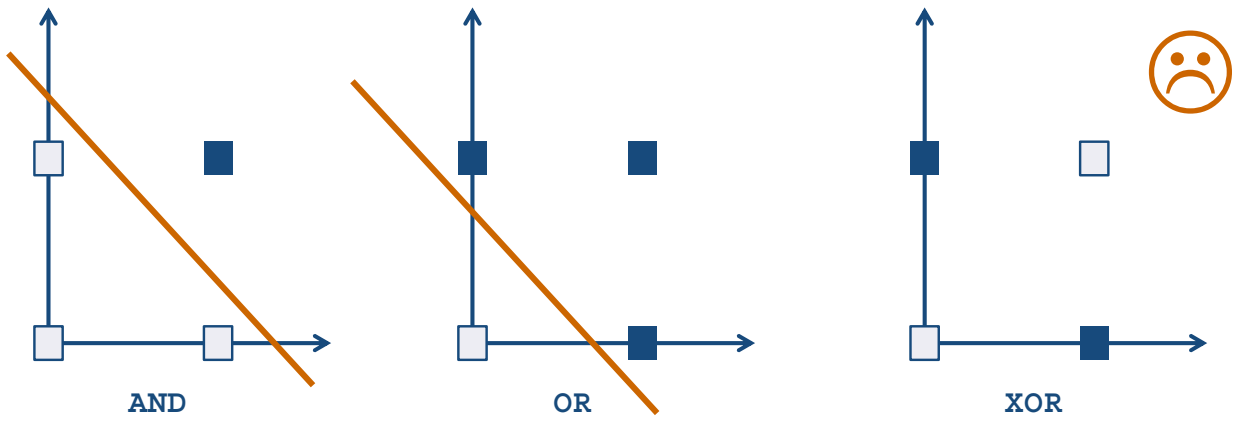
2- Calculer les valeurs de poids associées au modèle dans la 21ème itération

3- A quelle itération le processus d'ajustement de poids se converge ?

4- Déduire l'accuracy du modèle associé aux données d'apprentissage

	$\mathbf{n_1}$	$\mathbf{n_2}$	$\mathbf{n_3}$	**d**
0	2	0.8	1.2	Oui
1	0.7	1.9	0.1	Non
2	1.4	3.3	0.8	Oui
3	2.1	2.5	0.9	Non
4	1.1	2.9	2	Non
5	1.7	3	0.5	Oui
6	1.8	2.9	2	Oui
7	0.4	3	1.5	Non
8	2.1	2.8	1.4	Oui
9	1.6	0.3	2.7	Oui

Perceptron & Problème XOR!



Incapable de classer les données non linéairement séparables

Quiz

1. Le problème XOR peut être résolu en utilisant un perceptron simple (seule réponse)
   - ☐ Vrai
   - ☑ Faux
2. Étant donnée l'observation **z** (4 , 6 , 7 , **1**) de **3** trois descripteurs et une classe (0 ou 1), le passage de cette observation sur un perceptron ne change pas les poids sachant que **η =Tous les poids=1** (seule réponse)
   - ☑ Vrai
   - ☐ Faux
3. Combien il existe de paramètres à apprendre au total à partir d'un perceptron simple entrainé sur un dataset comportant 200 observations et 20 descripteurs ? (seule réponse)
   - ☐ 201
   - ☐ 200
   - ☑ 21
   - ☐ 20

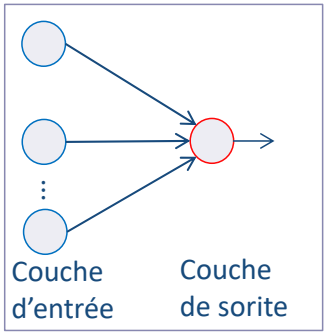
PERCEPTRON MULTICOUCHES

(FULLY CONNECTED)

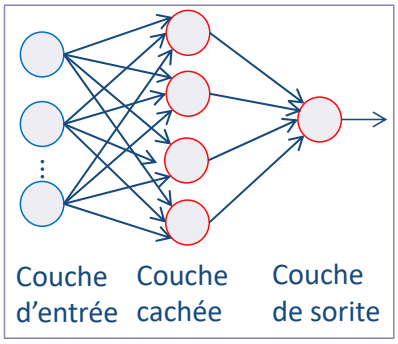
T. Ben Said & T. Ben Abdallah

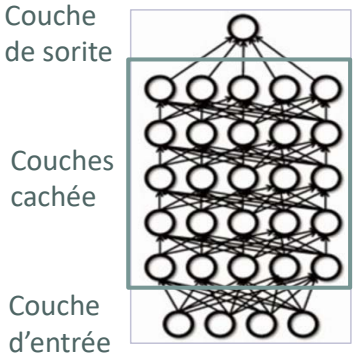
IIT-Sfax

Perceptron simple vs. multicouches



Perceptron simple



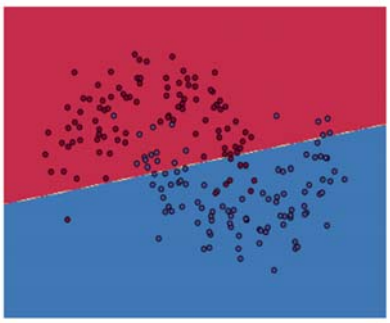


Perceptron multicouches (Multi-Layer Perceptron **MLP**)
= Fully connected

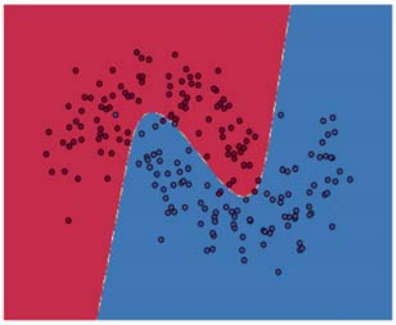
Le terme **"profond"** dans "apprentissage profond" fait référence au nombre de couches cachées du réseau de neurones

Impacts des couches cachées!

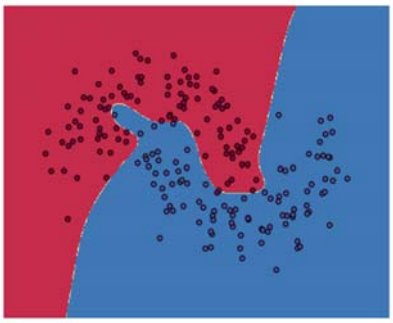
Pouvoir séparateur : permet de classer des formes **non linéairement séparables** ✅



0 Couche cachée



3 couches cachées
chacun de 10 neurones



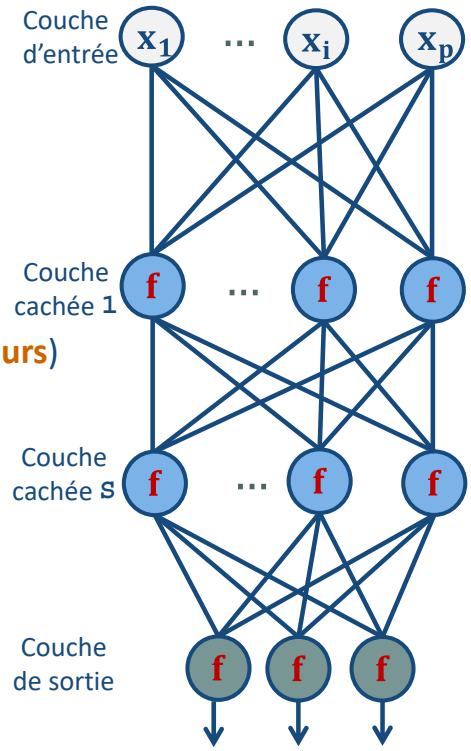
20 couches cachées
chacun de 100 neurones

↗ Nombre de couches
&
↗ Nombre de neurones

➔ Accroitre le pouvoir de séparation

Perceptron multicouches (MLP)

x_1	x_2	...	x_p	Classe_d
				C1
				C2
				C3
				. .



Couche d'entrée : p neurones (**Nombre de descripteurs**)

Une ou plusieurs couche(s) cachée(s)

Couche cachée 1 : k neurones

Couche cachée s : m neurones

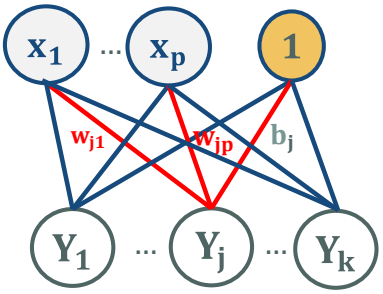
Fully Connected & Séquentiel

Couche de sortie : n neurones (**Nombre de classes**)

Fonctions d'activation : f

Fonction d'activation Sigmoïde

Étant donné $\mathbf{p}$ neurones $(\mathbf{x_1}, \dots, \mathbf{x_p})$ dans une couche et $\mathbf{k}$ neurones $(\mathbf{Y_1}, \dots, \mathbf{Y_j}, \mathbf{Y_k})$ dans la couche suivante

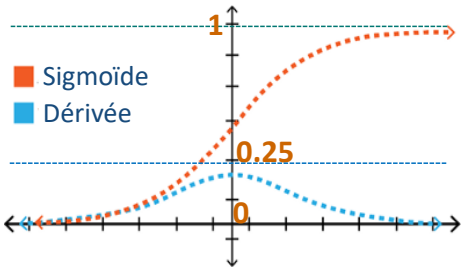


$$Y_j = \sum_{i=1}^{p} w_{ji} x_i + b_j$$

Sigmoïde $f(Y_j) = \frac{1}{1+e^{-Y_j}}$ $]0,1[$

Dérivée de la fonction Sigmoïde

$$\frac{\partial f(Y_j)}{\partial Y_j} = f'(Y_j) = f(Y_j)(1 - f(Y_j)) \quad]0,0.25[$$

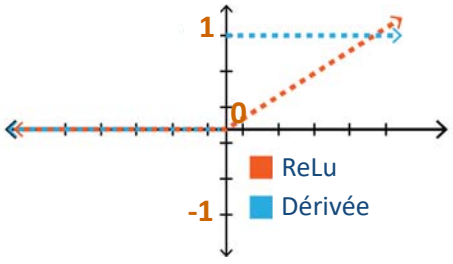


⚠ **Classification binaire**

Autres fonctions d'activations ReLU & ses variantes 1/2

Rectified Linear Unit (ReLU)

$$f(Y_j) = \max(0, Y_j) = \begin{cases} 0 & \text{si } Y_j < 0 \\ Y_j & \text{si } Y_j \geq 0 \end{cases} \qquad [0, +\infty[$$

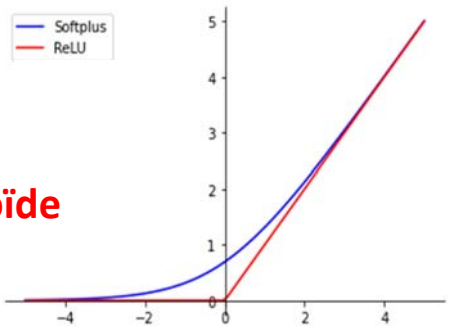


Dérivée de la fonction ReLu

$$\frac{\partial f(Y_j)}{\partial Y_j} = f'(Y_j) = \begin{cases} 0 & \text{si } Y_j < 0 \\ 1 & \text{si } Y_j \geq 0 \end{cases} \qquad 0 \text{ ou } 1$$

Softplus

$$f(Y_j) = \ln(e^{Y_j} + 1) \qquad]0, +\infty[$$

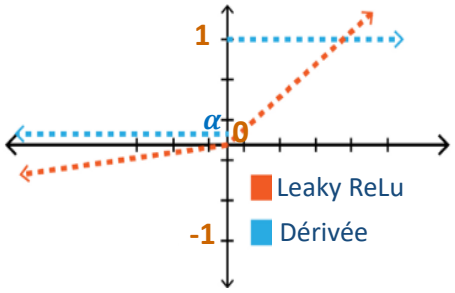


Dérivée de la fonction softplus= fonction Sigmoïde

$$\frac{\partial f(Y_j)}{\partial Y_j} = f'(Y_j) = \frac{1}{1+e^{-Y_j}} \qquad]0,1[$$

Autres fonctions d'activations ReLU & ses variantes 2/2

Leaky ReLu $f(Y_j)=\begin{cases}\alpha Y_j & si\ Y_j < 0\\ Y_j & si\ Y_j \geq 0\end{cases}$ $]-\infty,+\infty[$

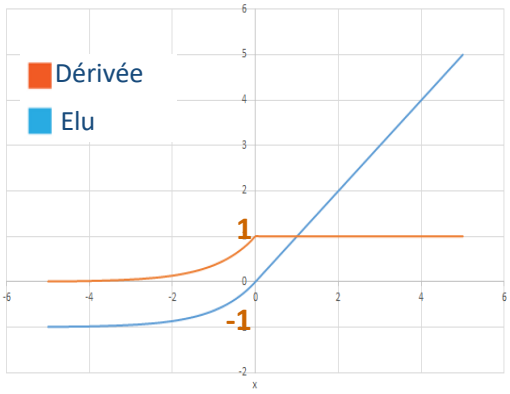


Dérivée de la fonction Leaky ReLu

$\frac{\partial f(Y_j)}{\partial Y_j}=f'(Y_j)=\begin{cases}\alpha & si\ Y_j < 0\\ 1 & si\ Y_j \geq 0\end{cases}$ α ou 1

Exponential Linear Unit (ELu)

$f(Y_j)=\begin{cases}\alpha\,(e^{Y_j}-1) & si\ Y_j < 0\\ Y_j & si\ Y_j \geq 0\end{cases}$ $]-\alpha,+\infty[$

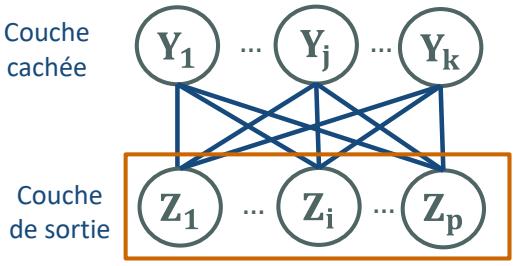


Dérivée de la fonction ELu

$\frac{\partial f(Y_j)}{\partial Y_j}=f'(Y_j)=\begin{cases}f(Y_j)+\alpha & si\ Y_j < 0\\ 1 & si\ Y_j \geq 0\end{cases}$ $]-\infty,1]$

Autres fonctions d'activations Softmax

Softmax $f(Z_i)=\frac{e^{Z_i}}{\sum_{a=1}^{p} e^{Z_a}}$ $[0,1]^p$ (somme=1)



Dérivée de la fonction Softmax

$f'(Z_i)=\begin{cases}f(Z_i)(1-f(Z_i)) & si\ i=j\\ -f(Z_i)f(Z_j) & si\ i\neq j\end{cases}$

☑ Utilisée pour activer les neurones de la couche de sortie

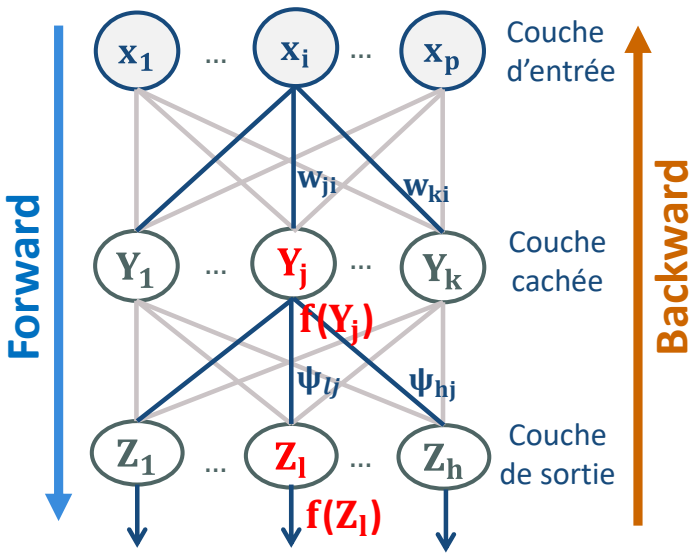
⚠ **Classification multi-classes**

Apprentissage : descente de gradient

1. **Forward Propagation**
   - *Transmettre les données tout au long du réseau*

2. **Backward Propagation**
   - *Définir une fonction de perte ou de coût (Loss E)*
   - *Transmettre l'erreur dans le sens inverse du réseau*
   - *Mettre à jour les paramètres W et b*

Apprentissage : descente de gradient



Forward

Transmission des données

$$f(Y_j) = f(\sum_{i=1}^{p} w_{ji}\ x_i + b_j)$$

$$f(Z_l) = f(\sum_{j=1}^{k} \psi_{lj}\ f(Y_j) + b_l)$$ =sortie

Backward

Définir E

Minimiser E & Ajuster les poids

$$w = w - \eta \frac{\partial E}{\partial w}$$

Fonction de perte (loss E)

Classification

Cross Entropy h classes (h>2) ➔ h neurones dans la couche de sortie

$$E = \sum_{l=1}^{h} -d_l \ln(f(Z_l))$$

avec
- h= Nombre de classe
- d= Classe désirée

Binary Cross Entropy 2 classes ➔ 1 neurone dans la couche de sortie Z

$$E = -[d \ln(f(Z)) + (1-d) \ln(1-f(Z))]$$

Régression

Mean Square Error 1 neurone dans la couche de sortie Z

$$E = \frac{1}{M} \sum_{i=1}^{M} (d^{(i)} - f(Z)^{(i)})^2$$

avec
- M= Nombre d'observation
- $d^{(i)}$= Valeur désirée de l'observation i
- $f(Z)^{(i)}$= Valeur prédite de l'observation i

Descente de gradient

Répéter

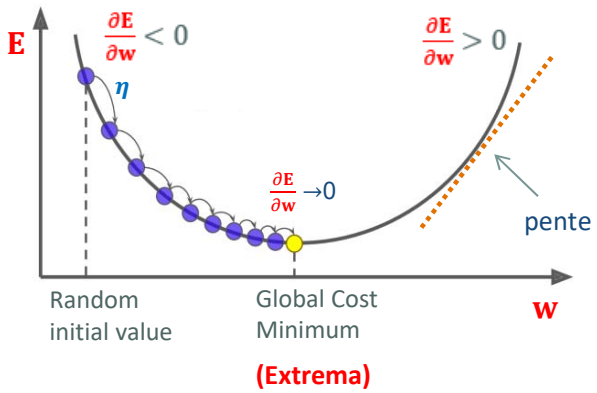
a. calculer le gradient (pente)

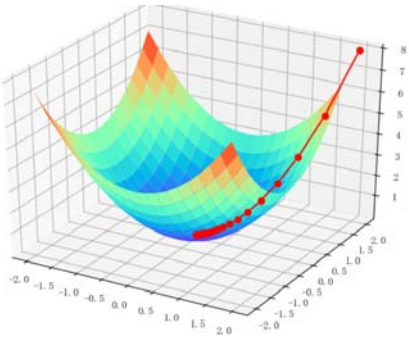
$$\nabla g = \frac{\partial E}{\partial w}$$

b. Mettre à jour les poids

$$w = w - \eta \frac{\partial E}{\partial w}$$

Jusqu'à Trouver le minimum d'erreur (convergence)





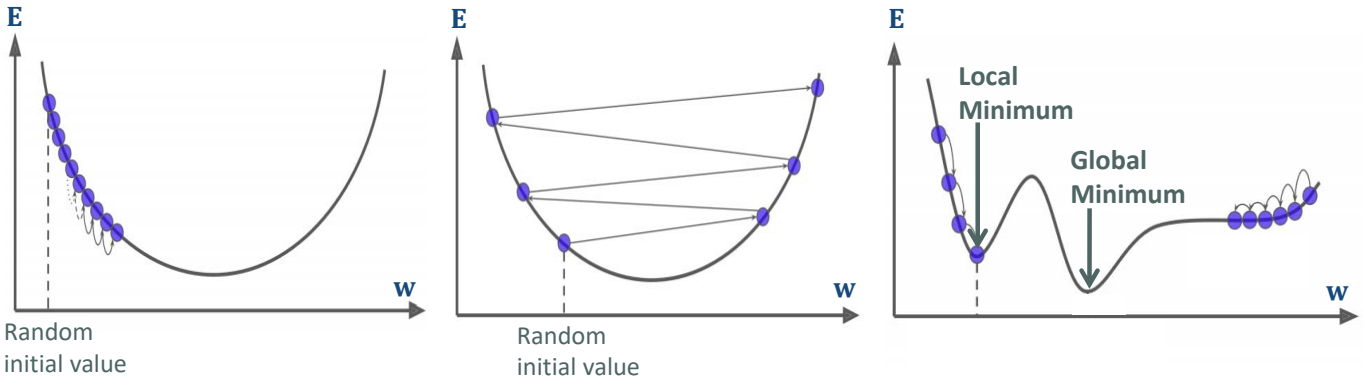
$\frac{\partial E}{\partial w} < 0$: la fonction E ↘ quand w ↗

➔ augmenter w

$\frac{\partial E}{\partial w} > 0$: la fonction E ↗ quand w ↗

➔ diminuer w

Descente de gradient : problèmes



η EST TROP PETIT

☹ Nombreuses itérations pour converger

☹ Temps important de calcul

η EST TROP ÉLEVÉ

☹ Sauter d'un côté à un autre

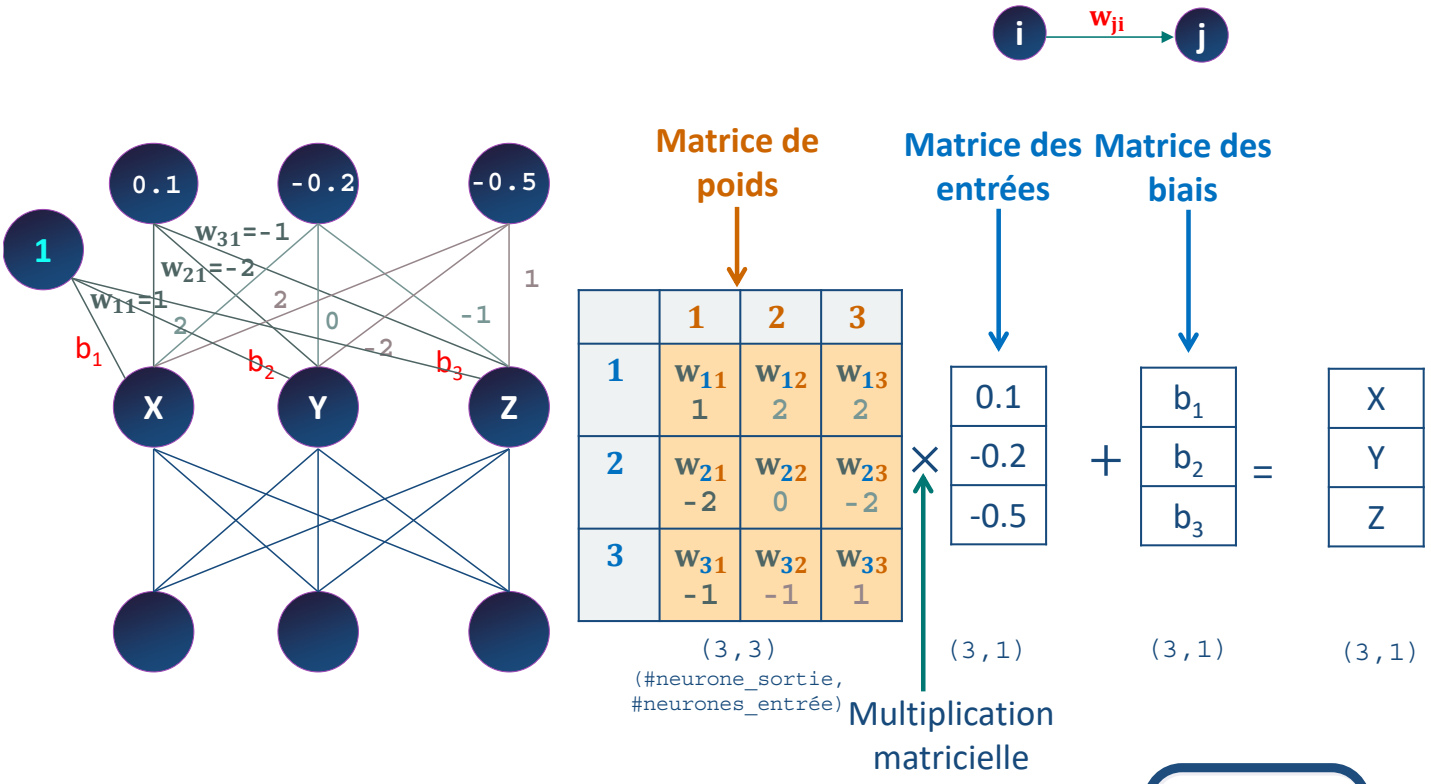
☹ Diverger l'algorithme

TOMBER DANS LE MINIMUM LOCAL

☹ Local Minimum>Global Minimum

Choix Learning rate η =0.001

Formulation mathématique



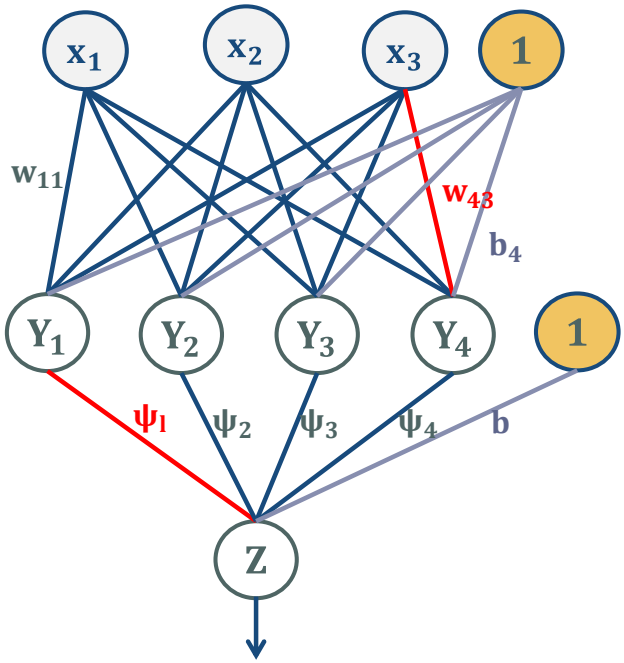
Activité (cas de classification binaire) 1/4

1. Modéliser un réseau de neurones avec une seule cachée de **4** neurones conformément au dataset suivant:

x_1	x_2	x_3	y
⋮	⋮	⋮	0
⋮	⋮	⋮	1
⋮	⋮	⋮	⋮
⋮	⋮	⋮	0
⋮	⋮	⋮	1

$$W = \begin{pmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \\ W_{31} & W_{32} & W_{33} \\ W_{41} & W_{42} & W_{43} \end{pmatrix} \quad B = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix}$$

$$\psi = (\psi_1 \quad \psi_2 \quad \psi_3 \quad \psi_4) \quad C = (b)$$



2. Donner les calculs nécessaires pour faire passer une observation dans ce modèle, puis mettre à jour les paramètres

Activité (cas de classification binaire) 2/4

I- Forward (propagation avant)

1- Calcul de l'activation des neurones de la couche cachée

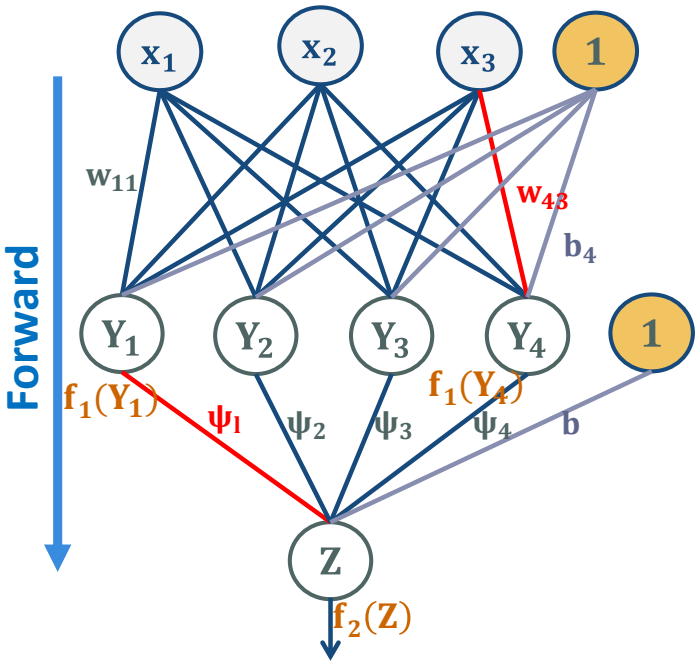
$$Y_j = \sum_{i=1}^{3} w_{ji}\ x_i + b_j$$

$$f_1(Y_j) = ReLU(Y_j) = max(0, Y_j)$$

2- Calcul de l'activation du neurone de sortie

$$Z = \sum_{j=1}^{4} \psi_j\ f_1(Y_j) + b$$

$$f_2(Z) = \sigma(Z) = \frac{1}{1 + e^{-Z}} = \text{sortie}$$



Activité (cas de classification binaire) 3/4

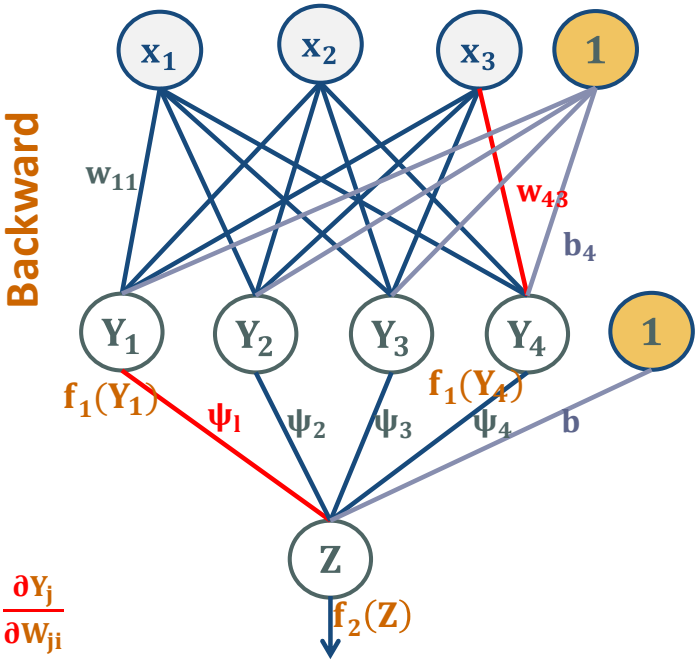
II- Backward : Descente de gradient

1- Calcul de loss E (Binary cross Entropy)

$$E = -[y \ln(f_2(Z)) + (1-y) \ln(1-f_2(Z))]$$

2- Calcul des gradients

$$\frac{\partial E}{\partial \psi_j} \qquad \frac{\partial E}{\partial W_{ji}}$$



Chain Rule

$$\frac{\partial E}{\partial \psi_j} = \frac{\partial E}{\partial f_2(Z)} \times \frac{\partial f_2(Z)}{\partial Z} \times \frac{\partial Z}{\partial \psi_j}$$

$$\frac{\partial E}{\partial W_{ji}} = \frac{\partial E}{\partial f_2(Z)} \times \frac{\partial f_2(Z)}{\partial Z} \times \frac{\partial Z}{\partial f_1(Y_j)} \times \frac{\partial f_1(Y_j)}{\partial Y_j} \times \frac{\partial Y_j}{\partial W_{ji}}$$

Activité (cas de classification binaire) 4/4

II- Backward : Descente de gradient

$$\frac{\partial E}{\partial \psi_j} = \frac{\partial E}{\partial f_2(Z)} \times \frac{\partial f_2(Z)}{\partial Z} \times \frac{\partial Z}{\partial \psi_j}$$

$$\frac{\partial Z}{\partial \psi_j} = f_1(Y_j)$$

$$f_2'(Z) = f_2(Z)(1 - f_2(Z))$$

$$\frac{\partial E}{\partial f_2(Z)} = -\frac{y}{f_2(Z)} + \frac{1-y}{1-f_2(Z)}$$

… …

$$E = -[y \ln(f_2(Z)) + (1-y) \ln(1-f_2(Z))]$$

$$\frac{\partial \ln(x)}{\partial X} = \frac{1}{x}$$

$$\frac{\partial \ln(1-x)}{\partial X} = -\frac{1}{1-x}$$

3- Mise à jour de tous les poids

Epoch, batch_size & Iteration



ÉPOQUE (epoch)

Passage sur l'ensemble de toutes les **n** observations de la base d'apprentissage



Batch_Size

Nombre total d'observations d'apprentissage présents dans un seul batch



Iteration

Nombre des batches nécessaires pour compléter une époque

*Batch ≠ Batch_Size!

Variantes de descente de gradient (1/2)

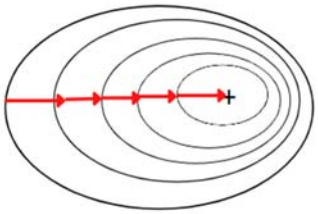
Descente de gradient par lot (classique) : Batch Gradient Descent (BGD)

- Mettre à jour les poids après avoir fait passer la totalité des observations par époque

$$w_i = w_i - \eta \cdot \frac{1}{N} \sum_{j=1}^{N} \frac{\partial E}{\partial w_i}$$

N= Nombre d'observation

☺ Stable
☹ Mise à jour lent des poids
☹ Facile à tomber dans le minimum local

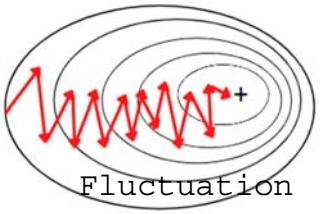


Descente de gradient stochastique : Stochastic Gradient Descent (SGD)

- Mettre à jour les poids au passage de chaque observation (indépendamment du nombre d'époque)

$$w_i = w_i - \eta \cdot \frac{\partial E}{\partial w_i}$$

☺ Mise à jour rapide des poids
☹ Instable
☹ Difficile de converger vers l'extrema

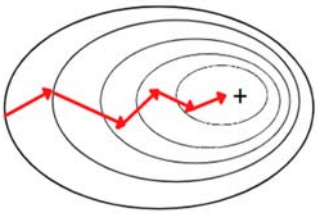


Variantes de descente de gradient (2/2)

☑ **Descente de gradient par Mini-lot :** Mini-Batch Gradient Descent (MBGD)

— Mettre à jour par blocs d'observations (batch-size)

$$w_i = w_i - \eta \cdot \frac{1}{M} \sum_{m=1}^{M} \frac{\partial E_m}{\partial w_i}$$



M= Nombre d'observation dans un batch=batch-size

☺ Convergence plus rapide que BGD
☹ Sensibilité à la taille du lot (batch_size)
☹ Facile à tomber dans le minimum local



Quiz

1. L'erreur Binary cross entropy est adaptée aux problèmes de classification binaire (seule réponse)

- ☑ Vrai
- ☐ Faux

2. Lesquelles des affirmations suivantes sont correctes? (réponse multiple)

- ☐ L'algorithme de descente de gradient classique (BGD) est rapide et fiable
- ☑ L'algorithme BGD risque tomber dans le local minimum
- ☑ L'algorithme de descente de gradient stochastique est rapide mais instable
- ☑ Le traitement par lot accélère la convergence par rapport à la descente de gradient classique (BGD) grâce à une mise à jour fréquente des poids

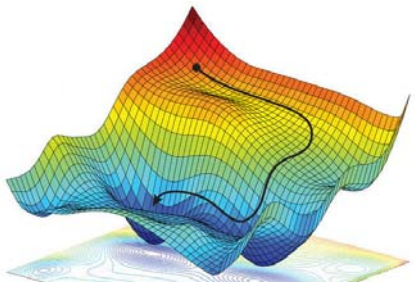
3. Étant donné une base d'apprentissage de 100 observations avec la taille d'un batch est égal à 1, alors (seule réponse)

- ☐ Il faudra une seule itération pour compléter une époque
- ☐ Il faudra deux itérations pour compléter une époque
- ☑ Il faudra 100 itérations pour compléter une époque
- ☐ Aucune réponse n'est correcte

Autres variantes de descente de gradient Optimiseur

☑ Problèmes du paysage d'optimisation

- Non convexité
- Minima locaux
- Points selles
- Plateaux
- Gradients faibles



☑ Problèmes de la méthode de descente de gradient

- Oscillations
- Convergence lente
- Learning rate mal adapté
- Gradients de différentes amplitudes selon les paramètres

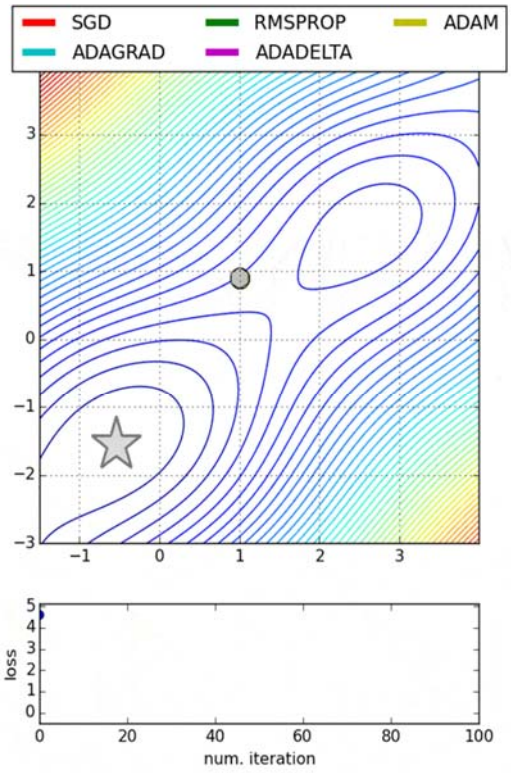
Autres variantes de descente de gradient Optimiseur

- **SGD avec Momentum**
  - Ajout d'une mémoire des gradients précédents
  - Réduction des oscillations
  - Accélération dans les directions cohérentes
  - Convergence rapide mais nécessité de la définition d'un nouveau hyperparamètre momentum
- **AdaGrad**
  - Taux d'apprentissage adapté à chaque paramètre
  - Accumulation des carrées des gradients
  - Coûteux en termes de calcul avec un taux d'apprentissage effectif qui diminue progressivement
- **RMSProp**
  - Amélioration de AdaGrad
  - Remplacement de l'accumulation par une moyenne glissante
  - Les gradients anciens perdent progressivement leur influence et le learning rate ne diminue plus systématiquement

Autres variantes de descente de gradient Optimiseur

- **AdaDelta**
  - Évolution de RMSProp
  - Adaptation des mises à jour basée sur les gradients récents et les mises à jour précédents
  - Aucune nécessité d'un taux d'apprentissage global

- **Adam (Adaptive Moment Estimation)**
  - Combinaison des deux idées :
    - Momentum : moyenne des gradients
    - RMSProp : adaptation du taux d'apprentissage pour chaque paramètre
  - Convergence très rapide mais coûteux en termes de calcul avec nécessité de définir deux hyperparamètres



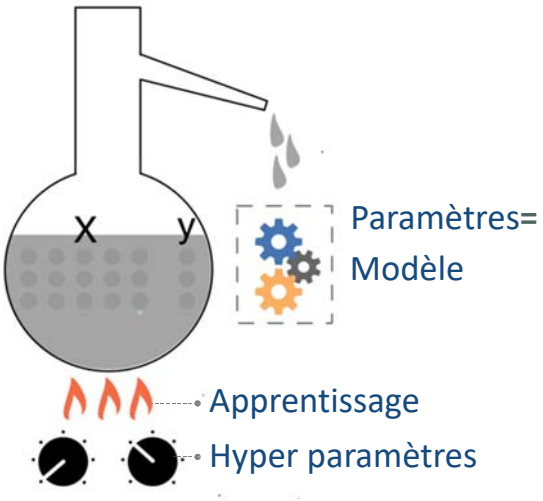
Pour résumer Paramètres & Hyper paramètres!

Paramètres =automatiquement appris par les modèles

- Poids
- Biais

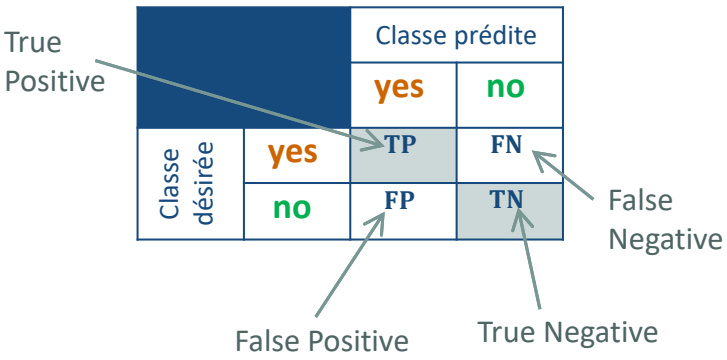
Hyper paramètres=configurés manuellement

- Learning rate η
- Nombre d'époques
- Taille d'un Batch
- Nombre de neurones dans chaque couche cachée
- Nombre de couches cachées
- Choix d'Optimiseur



Evaluation d'un réseau de neurones

CM		Classe prédite			
		C_1	...	C_i	C_N
Classe désirée	C_1	CM(1,1)		CM(1,i)	
	⋮		⋮		
	C_i			CM(i,i)	
	C_N				CM(N,N)



$$\text{Accuracy} = \frac{\sum_{i=1}^{N} CM(i,i)}{n}$$

$$\text{recall}_{C_i} = \frac{CM(i,i)}{\sum_{j=1}^{N} CM(i,j)}$$

$$\text{precision}_{C_i} = \frac{CM(i,i)}{\sum_{j=1}^{N} CM(j,i)}$$

$$F1 - \text{score}_{C_i} = \frac{2 \times (\text{precision}_{C_i} \times \text{recall}_{C_i})}{\text{precision}_{C_i} + \text{recall}_{C_i}}$$

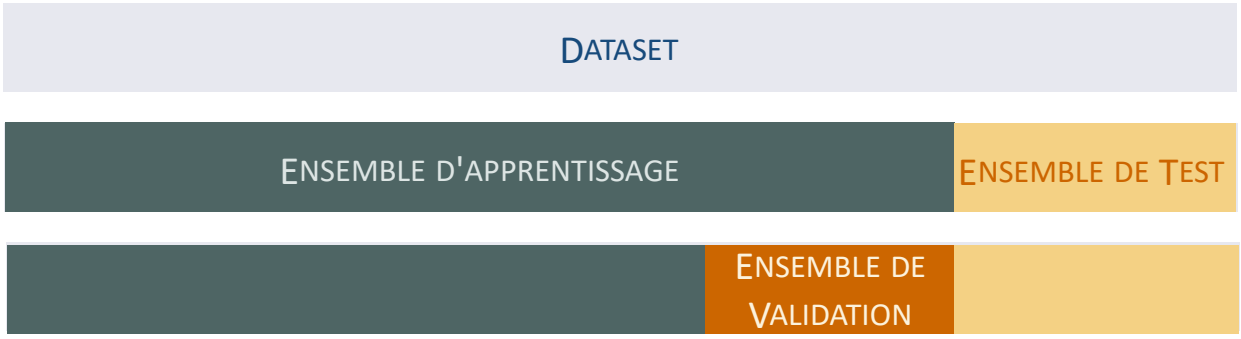
Cas de 2 classes

recall = **True Positive rate**

= Sensitivity = $\frac{TP}{TP+FN}$

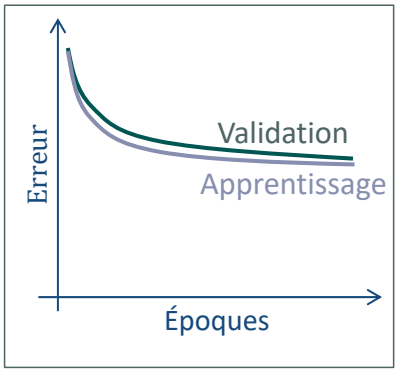
True Negative rate = Specificity = $\frac{TN}{FP+TN}$

Échantillonnage!

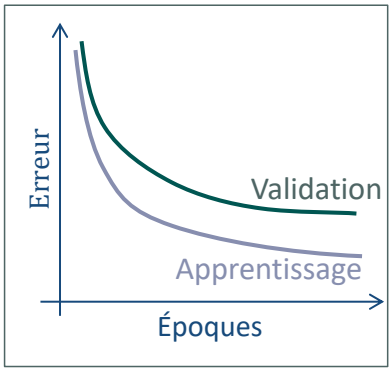


ENSEMBLE DE VALIDATION= utilisé pour determiner les valeurs des **hyperparamètres** du modèle ✓

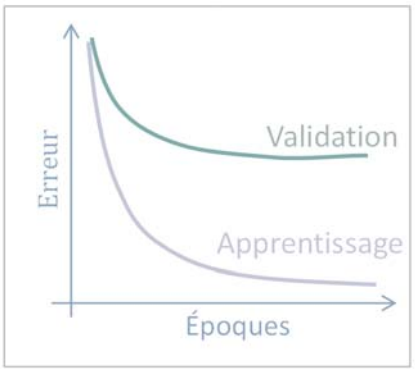
Sous-apprentissage vs. Sur-apprentissage



Sous-apprentissage
(Underfitting)



Parfait
(perfect)



Sur-apprentissage
(Overfitting)

TensorFlow - Keras

Cycle de vie du modèle

Définition
- Type, couches, activation

Compilation (.compile)
- Optimizer, loss, metrics

Entraînement (.fit)
- x, y, epochs, batch_size, validation_data

Évaluation (.evaluate)
- Test_data

Prédiction (.predict)
- New data